dYdX v4 기술 스택 및 의존성 분석

> **작성일**: 2025-01-08

> **분석 대상**: dYdX v4 Chain

> **작성자**: dYdX v4 구현체 분석팀

개요 (Overview)

기술 스택 역할과 책임

dYdX v4는 현대적인 블록체인 기술 스택을 활용하여 고성능 탈중앙화 거래소를 구현합니다. Protocol Layer는 Go 언어와 Cosmos SDK를 기반으로 하며, Indexer Layer는 TypeScript/Node.js 기반의 마이크로서비스 아키텍처를 채택합니다.

기술 선택의 중요성

각 기술 스택은 특정 요구사항을 충족하기 위해 신중하게 선택되었습니다:

- **Go**: 높은 성능과 동시성 처리를 위한 Protocol Layer
- **TypeScript/Node.js**: 빠른 개발과 유연성을 위한 Indexer Layer
- **PostgreSQL**: 복잡한 쿼리와 ACID 트랜잭션 지원
- **Kafka**: 고성능 이벤트 스트리밍과 확장성

Protocol Layer 기술 스택

핵심 프레임워크

Go 언어 (1.22)

- **역할**: Protocol Layer의 주 개발 언어
- **선택 이유**:
- 높은 성능과 메모리 효율성
- 강력한 동시성 지원 (goroutines)
- 정적 타입 시스템으로 안정성 확보
- 크로스 플랫폼 컴파일 지원

Cosmos SDK (v0.50.4)

- **역할**: 블록체인 애플리케이션 프레임워크
- **주요 기능**:
- 모듈화된 아키텍처
- ABCI (Application Blockchain Interface) 구현
- 상태 관리 및 트랜잭션 처리
- IBC (Inter-Blockchain Communication) 지원

```
// Cosmos SDK 모듈 구조 예시
type AppModule interface {
Name() string
RegisterInvariants(ir sdk.InvariantRegistry)
Route() sdk.Route
QuerierRoute() string
LegacyQuerierHandler(*codec.LegacyAmino) sdk.Querier
}
```

CometBFT (v0.38.5)

- **역할**: 합의 엔진 및 네트워킹 레이어
- **특징**:
- 비잔틴 장애 허용 (BFT) 합의 알고리즘
- 즉시 완결성 (Instant Finality)
- P2P 네트워킹 프로토콜
- dYdX 커스텀 포크 사용

주요 의존성

데이터 처리 및 저장

```
// 주요 의존성 목록
require (
cosmossdk.io/math v1.3.0 // 수학 연산
github.com/cosmos/cosmos-db v1.0.2 // 데이터베이스 추상화
github.com/cosmos/iavl v1.1.1 // IAVL 트리 (상태 저장)
github.com/syndtr/goleveldb v1.0.1 // LevelDB 구현
)
```

메시지 큐 및 통신

```
require (
github.com/Shopify/sarama v1.37.2 // Kafka 클라이언트
google.golang.org/grpc v1.62.0 // gRPC 통신
```

```
github.com/gorilla/websocket v1.5.1 // WebSocket 지원
)
```

암호화 및 보안

```
require (
github.com/cosmos/go-bip39 v1.0.0 // BIP39 니모닉
filippo.io/edwards25519 v1.0.0 // Ed25519 서명
github.com/ethereum/go-ethereum v1.12.0 // 이더리움 호환성
)
```

개발 도구

테스팅 및 품질 관리

```
require (
github.com/stretchr/testify v1.8.4 // 테스트 프레임워크
github.com/golang/mock v1.6.0 // 모킹 라이브러리
github.com/golangci/golangci-lint v1.56.2 // 린터
)
```

모니터링 및 디버깅

```
require (
gopkg.in/DataDog/dd-trace-go.v1 v1.48.0 // Datadog 트레이싱
```

```
    github.com/rs/zerolog  v1.32.0  // 구조화된 로깅

    github.com/prometheus/client_golang  v1.18.0  // 메트릭 수집

    )
```

Indexer Layer 기술 스택

핵심 런타임

Node.js (18+)

- **역할**: JavaScript/TypeScript 런타임 환경
- **특징**:
- 비동기 I/O 처리에 최적화
- 풍부한 npm 생태계
- 빠른 개발 사이클

TypeScript (4.7.4)

- **역할**: 정적 타입 JavaScript 슈퍼셋
- **장점**:
- 컴파일 타임 타입 검사
- 향상된 IDE 지원
- 대규모 프로젝트 관리 용이성

웹 프레임워크 및 API

Express.js (4.18.1)

- **역할**: HTTP 서버 프레임워크 (Comlink 서비스)

- **주요 미들웨어**:

```
// Express 미들웨어 구성 예시
app.use(cors()); // CORS 처리
app.use(bodyParser.json()); // JSON 파싱
app.use(responseTime()); // 응답 시간 측정
app.use(expressRequestId()); // 요청 ID 생성
```

TSOA (5.1.1)

- **역할**: TypeScript 기반 OpenAPI/Swagger 생성
- **기능**:
- 자동 API 문서 생성
- 런타임 검증
- 타입 안전성 보장

데이터베이스 및 캐싱

PostgreSQL (14+)

- **역할**: 주 데이터베이스
- **선택 이유**:
- ACID 트랜잭션 지원
- 복잡한 쿼리 최적화
- JSON 데이터 타입 지원
- 강력한 인덱싱 기능

Redis (5.0.6)

- **역할**: 캐싱 및 세션 저장소
- **사용 사례**:
- API 응답 캐싱
- Rate limiting
- 세션 관리

```
// Redis 연결 설정 예시
const redis = require('redis');
const client = redis.createClient({
url: process.env.REDIS_URL || 'redis://localhost:6379'
});
```

메시지 큐 및 이벤트 처리

Apache Kafka (2.6)

- **역할**: 분산 이벤트 스트리밍 플랫폼
- **토픽 구조**:

```
# Kafka 토픽 설정
KAFKA_CREATE_TOPICS:
- "to-ender:1:1"  # 온체인 이벤트
- "to-vulcan:1:1"  # 오프체인 이벤트
- "to-websockets-orderbooks:1:1"  # 주문서 WebSocket
- "to-websockets-subaccounts:1:1"  # 계정 WebSocket
```

```
- "to-websockets-trades:1:1"  # 거래 WebSocket
- "to-websockets-markets:1:1"  # 시장 WebSocket
- "to-websockets-candles:1:1"  # 캔들 WebSocket
```

KafkaJS (2.1.0)

- **역할**: Node.js Kafka 클라이언트
- **기능**:
- 프로듀서/컨슈머 구현
- 컨슈머 그룹 관리
- 자동 재연결 및 오류 처리

서비스별 주요 의존성

Comlink (HTTP API 서비스)

```
{
"dependencies": {
"express": "^4.18.1", // HTTP 서버
"cors": "^2.8.5", // CORS 처리
"express-validator": "^6.14.2", // 입력 검증
"swagger-ui-express": "^4.6.2", // API 문서화
"rate-limiter-flexible": "^2.4.1", // Rate limiting
"big.js": "^6.2.1", // 정밀 수치 계산
"luxon": "^3.0.1"  // 날짜/시간 처리
}
```

```
}
```

Ender (온체인 데이터 아카이브)

```
{
"dependencies": {
"kafkajs": "^2.1.0", // Kafka 클라이언트
"pg": "^8.7.3", // PostgreSQL 클라이언트
"long": "^5.2.1", // 64비트 정수 처리
"@dydxprotocol-indexer/v4-proto-parser": "workspace:^0.0.1"
}
}
```

외부 의존성 분석

데이터베이스 시스템

PostgreSQL 설정

```
# Docker Compose 설정
postgres:
image: postgres:14
environment:
POSTGRES_PASSWORD: dydxserver123
```

```
POSTGRES_USER: dydx_dev

ports:

- "5435:5432"

healthcheck:

test: ["CMD-SHELL", "pg_isready -U dydx_dev"]

interval: 5s

timeout: 20s

retries: 10
```

주요 PostgreSQL 확장

- **pg_stat_statements**: 쿼리 성능 분석
- **pg_trgm**: 텍스트 검색 최적화
- **btree_gin**: 복합 인덱스 지원

컨테이너화 및 오케스트레이션

Docker

- **Protocol**: 멀티스테이지 빌드로 최적화
- **Indexer**: 서비스별 개별 컨테이너
- **의존성**: PostgreSQL, Kafka, Redis 컨테이너화

```
# Protocol Dockerfile 예시

FROM golang:1.22-alpine AS builder

WORKDIR /app

COPY go.mod go.sum ./
```

```
RUN go mod download

COPY . .

RUN go build -o dydxprotocold ./cmd/dydxprotocold



FROM alpine:latest

RUN apk --no-cache add ca-certificates

COPY --from=builder /app/dydxprotocold /usr/local/bin/

ENTRYPOINT ["dydxprotocold"]
```

Docker Compose

- **로컬 개발**: 전체 스택 일괄 실행
- **서비스 의존성**: 헬스체크 기반 순차 시작
- **네트워킹**: 서비스 간 내부 네트워크 구성

모니터링 및 관찰성

Datadog

- **APM**: 애플리케이션 성능 모니터링
- **로그 수집**: 구조화된 로그 중앙 집중화
- **메트릭**: 시스템 및 비즈니스 메트릭 수집

```
// Datadog 트레이싱 설정

import tracer from 'dd-trace';

tracer.init({
```

```
service: 'comlink',
env: process.env.DD_ENV,
profiling: true,
runtimeMetrics: true
});
```

로깅 시스템

```
// Protocol 로깅 (zerolog)
logger := zerolog.New(os.Stdout).With().
Timestamp().
Str("service", "dydxprotocol").
Logger()

logger.Info().
Str("module", "clob").
Uint64("height", height).
Msg("Processing block")
```

개발 및 배포 환경 요구사항

로컬 개발 환경

Protocol 개발 요구사항

```
# 필수 도구
- Go 1.22+
- Docker Desktop
- Make
- Git

# 설치 및 실행
make install # 의존성 설치
make localnet-start # 로컬 네트워크 시작
make test # 테스트 실행
make lint # 코드 품질 검사
```

Indexer 개발 요구사항

```
# 필수 도구
- Node.js 18+
- pnpm 6+
- Docker Desktop
- PostgreSQL 14+

# 설치 및 실행
nvm use # Node.js 버전 설정
pnpm install # 의존성 설치
```

```
docker-compose -f docker-compose-local-deployment.yml up # 서비스 시작
pnpm run test:all # 테스트 실행
```

프로덕션 배포 환경

AWS 인프라 요구사항

- **컴퓨팅**: ECS Fargate 또는 EC2
- **데이터베이스**: RDS PostgreSQL (Multi-AZ)
- **캐싱**: ElastiCache Redis
- **메시지 큐**: Amazon MSK (Kafka)
- **로드 밸런서**: Application Load Balancer
- **모니터링**: CloudWatch + Datadog

환경 변수 설정

```
# Protocol 환경 변수
DAEMON_HOME=/dydxprotocol
PRICE_DAEMON_ENABLED=true
LIQUIDATION_DAEMON_ENABLED=true
BRIDGE_DAEMON_ENABLED=true
DD_ENV=production
DD_AGENT_HOST=datadog-agent

# Indexer 환경 변수
```

```
DATABASE_URL=postgresql://user:pass@rds-endpoint:5432/dydx

KAFKA_BROKER_URLS=kafka1:9092,kafka2:9092,kafka3:9092

REDIS_URL=redis://elasticache-endpoint:6379

DD_PROFILING_ENABLED=true

RATE_LIMIT_ENABLED=true
```

CI/CD 파이프라인

GitHub Actions 워크플로우

```yaml
# Protocol 테스트 및 빌드

name: Protocol Test

on: [push, pull_request]

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- uses: actions/setup-go@v3

with:

go-version: 1.22

- run: make test

- run: make lint
```

배포 자동화

- **이미지 빌드**: Docker 이미지 자동 빌드
- **ECR 푸시**: AWS ECR에 이미지 푸시
- **ECS 배포**: Terraform을 통한 인프라 관리
- **헬스체크**: 배포 후 서비스 상태 확인

성능 및 확장성 고려사항

Protocol Layer 최적화

Go 런타임 최적화

```
// 메모리 최적화 설정
runtime.GOMAXPROCS(runtime.NumCPU())
debug.SetGCPercent(100) // GC 튜닝
```

데이터베이스 최적화

- **LevelDB**: SSD 최적화 설정
- **IAVL 트리**: 메모리 캐싱 활용
- **상태 동기화**: 스냅샷 기반 빠른 동기화

Indexer Layer 최적화

Node.js 성능 튜닝

```
// 힙 메모리 최적화
```

```
node --max-old-space-size=4096 --heapsnapshot-signal=SIGUSR2


// 클러스터링
const cluster = require('cluster');
const numCPUs = require('os').cpus().length;


if (cluster.isMaster) {
for (let i = 0; i < numCPUs; i++) {
cluster.fork();
}
}
```

데이터베이스 연결 풀링

```
// PostgreSQL 연결 풀 설정
const pool = new Pool({
host: process.env.DB_HOST,
port: 5432,
database: process.env.DB_NAME,
user: process.env.DB_USER,
password: process.env.DB_PASSWORD,
max: 20, // 최대 연결 수
idleTimeoutMillis: 30000,
connectionTimeoutMillis: 2000,
```

```
});
```

보안 고려사항

Protocol Layer 보안

암호화 및 서명

- **Ed25519**: 디지털 서명 알고리즘
- **BIP39**: 니모닉 시드 생성
- **TLS**: 모든 네트워크 통신 암호화

합의 보안

- **검증자 다양성**: 지리적, 조직적 분산
- **슬래싱**: 악의적 행동에 대한 경제적 처벌
- **업그레이드 거버넌스**: 온체인 거버넌스를 통한 안전한 업그레이드

Indexer Layer 보안

API 보안

```
// Rate limiting 설정
const rateLimiter = new RateLimiterRedis({
storeClient: redisClient,
keyPrefix: 'api_limit',
points: 100, // 요청 수
duration: 60, // 시간 (초)
```

```
});


// 입력 검증
app.use(validator.body('amount').isNumeric());
app.use(validator.param('id').isUUID());
```

데이터 보안

- **연결 암호화**: TLS/SSL 강제
- **접근 제어**: 역할 기반 접근 제어 (RBAC)
- **감사 로그**: 모든 중요 작업 로그 기록

향후 기술 로드맵

단기 개선 사항

- [ ] Go 1.23 업그레이드
- [ ] Node.js 20 LTS 마이그레이션
- [ ] PostgreSQL 15 업그레이드
- [ ] Kafka 3.x 마이그레이션

장기 기술 전략

- [ ] WebAssembly (WASM) 스마트 컨트랙트 지원
- [ ] GraphQL API 도입
- [ ] 마이크로서비스 메시 (Istio) 적용

- [ ] 멀티 리전 배포 지원

참고 자료 (References)

- Go 공식 문서
- Cosmos SDK 문서
- CometBFT 문서
- Node.js 공식 문서
- TypeScript 핸드북
- PostgreSQL 문서
- Apache Kafka 문서

변경 이력 (Change History)

| 날짜 | 버전 | 변경 내용 | 작성자 |

|------|------|----------|--------|

| 2025-01-08 | 1.0 | 초기 작성 | dYdX v4 구현체 분석팀 |